

PD-3: Comments on events

Status: tickets approved · Revision 2 · Created by Ivan Blažević <ivan.blazevic@rubycode.co> · 30 September 2026

Overview

What

Signed-in people can leave a comment on an event page, to ask a question or share a link. Comments are listed oldest first, with the author's name and when it was posted. Authors can delete their own comments, and organizers can delete any comment on their event.

Why

Questions about events are asked in chat today and get lost. On the event page, the answer helps the next person with the same question.

Where

The event page, below the RSVP card.

Who

Team Gather. Ana Kovač reviews the pull requests.

Background

Existing Data Structure

The models and code below already exist. This change either touches them or copies their patterns. Everything the feature adds is new and is described under Database changes and Application changes.

Event: `app/models/event.rb` (table `events`)

Column	Type	Null
<code>id</code>	integer	not null
<code>title</code>	string	not null
<code>description</code>	text	null

Column	Type	Null
starts_at	datetime	not null
venue	string	not null
capacity	integer	not null
organizer_id	integer	not null, FK to users
created_at / updated_at	datetime	not null

Indexes: `index_events_on_organizer_id`, `index_events_on_starts_at`.

Associations:

- `belongs_to :organizer, class_name: "User"`
- `has_many :rsvps, dependent: :delete_all`
- `has_many :attendees, -> { merge(Rsvp.going) }, through: :rsvps, source: :user`

The method that matters here is `Event#organized_by?(user)`. It returns `user.present? && organizer_id == user.id`. The app uses it for every organizer check, both in `EventsController#require_organizer` and in the views.

User: `app/models/user.rb` (table `users`)

Column	Type	Null / default
id	integer	not null
email_address	string	not null, unique index
password_digest	string	not null
name	string	not null, default ""
created_at / updated_at	datetime	not null

Associations:

- `has_many :sessions, dependent: :destroy`
- `has_many :rsvps, dependent: :destroy`
- `has_many :organized_events, class_name: "Event", foreign_key: :organizer_id, inverse_of: :organizer, dependent: :destroy`

The model has `validates :name, presence: true`, but the column defaults to an empty string. Rows created before the validation existed could therefore have a blank `name`.

Rsvp: `app/models/rsvp.rb` (table `rsvps`)

This model does not change. It is the pattern to copy:

- It has `event_id` and `user_id` columns, both not null with foreign keys.
- It has a composite index `[event_id, status, created_at]`.
- It orders rows with `scope :in_line_order, -> { order(:created_at, :id) }`.

- It logs with `Rails.logger.info` using `key=value` pairs.

Session: `app/models/session.rb` (`table sessions`)

This model does not change. The `Authentication` concern

(`app/controllers/concerns/authentication.rb`) resolves `Current.session` and `Current.user` from it.

Related existing code

- `config/routes.rb`: `resources :events, except: :destroy do resource :rsvp, only: %i[ create destroy ] end`.
- `app/controllers/events_controller.rb`: `allow_unauthenticated_access only: %i[ index show ]` plus `resume_session`. `#show` sets `@rsvp`, and sets `@rsvps` only for the organizer.
- `app/controllers/rsvps_controller.rb`: a nested controller with `set_event` via `Event.find(params[:event_id])`. `require_authentication` redirects guests to sign-in.
- `app/views/events/show.html.erb`: `<aside class="card event__facts" id="event-facts">` renders `events/rsvp_card`, then the organizer's **Edit event** link, then `events/attendees`.
- `app/views/events/_rsvp_card.html.erb`: the card with `id="rsvp-card"`.
- `app/helpers/application_helper.rb`: `event_time(event)` formats times as `%A, %-d %B %Y`
• `%H:%M`.

Architectural changes

Target design

Comments are added as one new resource nested under events. The request flow stays the same as for RSVPs: a plain form post, a redirect back to `event_path`, and a flash message. There are no Turbo Streams, no background jobs and no broadcasting.

Creating a comment

1. A signed-in user submits the form on `events/show`, which sends `POST /events/:event_id/comments`.
2. `CommentsController#create` finds the event and builds `@event.comments.new(body:, user: Current.user)`.
3. On success it redirects to `event_path(@event, anchor: "comments")` with the notice `Comment posted`.
4. On validation failure it redirects to the event with the model's error as the alert. This keeps the controller from repeating the `EventsController#show` setup.

Deleting a comment

1. The user clicks a button that sends `DELETE /events/:event_id/comments/:id`.
2. The controller loads `@event.comments.find(params[:id])`. Because the lookup is scoped to the event, a comment id from another event returns 404.

3. It checks `comment.deletable_by?(Current.user)` . This is true for the author or for the event's organizer (`event.organized_by?(user)`).
4. It hard-deletes the comment and redirects with `Comment deleted.`

Reading comments

- `EventsController#show` loads `@comments = @event.comments.oldest_first.includes(:user)` .
- Anyone can read comments, including guests, because `show` already allows unauthenticated access.
- Guests see a `Sign in to comment` link instead of the form. This matches the RSVP card pattern.

Authorization

- Authorization is a model predicate, `Comment#deletable_by?(user)` .
- This follows the style of the existing `Event#organized_by?` . The app has no policy library (no Pundit or ActionPolicy in the `Gemfile`), and this plan doesn't add one.

How legacy data coexists during rollout

This is a new, empty table with no existing data to migrate. The expand, dual write, backfill, switch reads and contract sequence therefore reduces to **expand only**:

- **Expand**: create `comments` . Old code never reads it, so the migration can be deployed before or together with the new code.
- **Dual write and backfill**: not applicable. There is no legacy source of comments, and chat history is not imported.
- **Switch reads**: the new view partial reads from `comments` straight away.
- **Contract**: nothing to remove.

One piece of legacy data does need handling: `users.name` defaults to `""` , so older users may have a blank name. The comment partial must show a fallback such as `Someone` instead of an empty author line.

Deletion cascades

The app deletes events through `User has_many :organized_events, dependent: :destroy` . Deleting an event or a user must not fail on the new foreign keys:

- `Event has_many :comments, dependent: :delete_all` , the same as `rsvps` .
- `User has_many :comments, dependent: :delete_all` .

Database changes

New table: `comments`

This ships as one additive migration, `db/migrate/<timestamp>_create_comments.rb` . Generate it with `bin/rails g model Comment event:references user:references body:text` , then edit it to match the code below. It follows `db/migrate/20260929214624_create_rsvps.rb` , which also

passes `index: false` on `t.references :event` and declares its check constraint inside `create_table`.

```
class CreateComments < ActiveRecord::Migration[8.0]
  def change
    create_table :comments do |t|
      t.references :event, null: false, foreign_key: true, index: false
      t.references :user, null: false, foreign_key: true
      t.text :body, null: false

      t.timestamps

      t.index [ :event_id, :created_at ]
      t.check_constraint "length(trim(body)) > 0 AND length(body) <= 1000", name: "comments_body_length_check"
    end
  end
end
```

Use the same `ActiveRecord::Migration[x.y]` version as the existing migrations.

Column	Type	Null	Default	Notes
<code>id</code>	integer	not null	auto	Primary key.
<code>event_id</code>	integer	not null	none	Foreign key to <code>events.id</code> .
<code>user_id</code>	integer	not null	none	Foreign key to <code>users.id</code> .
<code>body</code>	text	not null	none	Between 1 and 1,000 characters, enforced by the model and by <code>comments_body_length_check</code> .
<code>created_at</code>	datetime	not null	none	Used for ordering and for "when it was posted".
<code>updated_at</code>	datetime	not null	none	

The database is SQLite (`gem "sqlite3"`), so the foreign key columns are `integer`, as in `rsvps`.

Indexes

- `index_comments_on_event_id_and_created_at` on `[event_id, created_at]`. This serves the event page query `WHERE event_id = ? ORDER BY created_at, id`. It replaces the plain `event_id` index that `t.references` would otherwise create, which is why `t.references :event` has `index: false`.
- `index_comments_on_user_id` on `user_id`. `t.references :user` creates this by default. It serves `user.comments.delete_all` when a user is deleted.

Foreign keys

- `comments.event_id` references `events.id`.
- `comments.user_id` references `users.id`.
- Neither has `on_delete`. Deletion cascades are handled in the application with `dependent: :delete_all` on `Event#comments` and `User#comments` (see Application changes), the same way `rsvps` works.

Check constraint: `comments_body_length_check`

The constraint is required, not optional. It follows the `rsvps_status_check` precedent in `db/schema.rb`.

- `length(trim(body)) > 0` rejects an empty or space-only body.
- `length(body) <= 1000` rejects a body longer than 1,000 characters. SQLite's `length()` counts characters on text values, not bytes, so accented names and emoji count the same way Ruby's `String#length` does.
- The constraint is safe to add because the table is created empty.

The model must use the same limit. `Comment::MAX_LENGTH` changes from `2_000` to `1_000`, and `validates :body, length: { maximum: MAX_LENGTH }` uses it. The textarea's `maxlength: Comment::MAX_LENGTH` picks up the new value automatically. The model validation is what users actually hit, because it returns a readable error instead of raising `ActiveRecord::StatementInvalid`. The constraint only catches writes that skip validations, such as `update_column`, `insert_all` or raw SQL.

The model and the database check blank bodies slightly differently. `normalizes :body` uses Ruby's `strip`, which removes all whitespace, including newlines and tabs. SQLite's `trim()` removes only spaces. The model check is the stricter of the two, so every body the model accepts also passes the constraint.

One edge case at the limit: browsers count a line break as one character for `maxlength`, but they submit it as `\r\n`, which is two characters. A multi-line comment close to 1,000 characters can therefore pass the browser check and then fail the model validation. It fails with an alert, not with a database error, but the typed text is lost on redirect. See Outstanding questions for whether to normalize `\r\n` to `\n`.

Changes to existing tables

None. `events`, `users`, `rsvps` and `sessions` are not altered. Nothing is removed or renamed, so no `ignored_columns` step is needed.

Rollout order (expand, dual write, backfill, switch reads, contract)

This is a new table with no legacy data, so only the expand step applies.

1. **Expand:** run the migration. The table is empty and the running code does not use it, so it can deploy before the application code or together with it.
2. **Dual write and backfill:** not applicable. There is no older source of comments, and chat history is not imported.

3. **Switch reads:** the new code reads from `comments` straight away.

4. **Contract:** nothing to remove.

If the limit changes later, the constraint must change in its own migration: remove `comments_body_length_check` and add it again with the new value. Raising the limit is safe.

Lowering it fails if existing rows are too long, so those rows have to be found and handled before the new constraint is added.

Rollback

`bin/rails db:rollback` drops `comments`, including its indexes and check constraint, and deletes every posted comment. Roll back the code first, and drop the table only if the feature is abandoned.

Application changes

Model: `app/models/comment.rb` (new)

```
class Comment < ApplicationRecord
  MAX_LENGTH = 2_000

  belongs_to :event
  belongs_to :user

  validates :body, presence: true, length: { maximum: MAX_LENGTH }

  normalizes :body, with: ->(body) { body.strip }

  scope :oldest_first, -> { order(:created_at, :id) }

  def deletable_by?(user)
    user.present? && (user_id == user.id || event.organized_by?(user))
  end
end
```

- `MAX_LENGTH` is an assumption (see Outstanding questions).
- The `id` tiebreaker in `oldest_first` keeps the order stable when two comments share a `created_at`, as `Rsvp.in_line_order` does.

Model: `app/models/event.rb`

Add `has_many :comments, dependent: :delete_all` next to `has_many :rsvps`. Nothing else changes.

Model: `app/models/user.rb`

Add `has_many :comments, dependent: :delete_all`. `delete_all` avoids loading every comment when a user is deleted. `Comment` has no callbacks that need to run.

Routes: `config/routes.rb`

```
resources :events, except: :destroy do
  resource :rsvp, only: %i[ create destroy ]
  resources :comments, only: %i[ create destroy ]
end
```

Controller: `app/controllers/comments_controller.rb` (new)

- **Authentication:** inherits `require_authentication` from `ApplicationController`. There is no `allow_unauthenticated_access`, so guests are redirected to `new_session_path`, the same as in `RsvpsController`.
- `before_action :set_event`: `@event = Event.find(params[:event_id])`. Unknown events return 404.
- `create`:
- `@comment = @event.comments.new(comment_params.merge(user: Current.user))`.
- If it saves, `redirect_to event_path(@event, anchor: "comments")`, notice: "Comment posted."
- Otherwise, `redirect_to event_path(@event, anchor: "comments")`, alert: `@comment.errors.full_messages.to_sentence`.
- `destroy`:
- `comment = @event.comments.find(params[:id])`.
- If `comment.deletable_by?(Current.user)` is false, `redirect_to event_path(@event)`, alert: "You can't delete this comment." This mirrors `require_organizer`.
- Otherwise call `comment.destroy!`, log it, and `redirect_to event_path(@event, anchor: "comments")`, notice: "Comment deleted."
- `comment_params`: `params.expect(comment: [ :body ])`. `user_id` and `event_id` are never taken from params.
- **Logging** (same style as `Rsvp#promote_next_waitlisted`): `Rails.logger.info("Comment deleted: comment_id=#{id} event_id=#{event_id} author_id=#{user_id} deleted_by=#{Current.user.id}")`.
- **Rate limiting (optional)**: `rate_limit to: 10, within: 1.minute, only: :create`, using the Rails 8 built-in. This depends on the production cache store (see Infrastructure changes and Outstanding questions).

Controller: `app/controllers/events_controller.rb`

In `#show`, add:

```
@comments = @event.comments.oldest_first.includes(:user)
```

The form must build `Comment.new`, not `@event.comments.build`. Building on the association would add an unsaved record to the loaded list.

Views

`app/views/events/show.html.erb` : render `events/comments` directly after render `"events/rsvp_card"` , as the team asked. Whether it sits inside the sidebar `aside` or below `.event__layout` is an open question.

`app/views/events/_comments.html.erb` **(new)**: a `<section class="section comments" id="comments">` containing:

- The heading `Comments (N)` , using `@comments.size` . The records are already loaded, so this runs no extra count query.
- `<ol class="comments__list" id="comments-list">` rendering `render partial: "comments/comment", collection: @comments` .
- When there are no comments: `<p class="muted small">No comments yet. Ask a question.</p>` .
- For signed-in users (`authenticated?`): `form_with model: [ @event, Comment.new ], id: "new-comment", with text_area :body, required: true, maxlength: Comment::MAX_LENGTH` and a submit button using `data: { turbo_submits_with: "Posting..." }` . `maxlength` keeps an over-long comment from being submitted and then lost on redirect.
- For guests: `link_to "Sign in to comment", new_session_path, class: "button"` .

`app/views/comments/_comment.html.erb` **(new)**: an `<li id="<%= dom_id(comment) %>">` containing:

- The author: `comment.user.name.presence || "Someone"` . The fallback covers legacy users with blank names.
- The time: `<time datetime="<%= comment.created_at.iso8601 %>"><%= comment_time(comment) %></time>` .
- The body: `simple_format(h(comment.body))` . The explicit `h` makes any HTML the user typed show as text. Without it, `simple_format` 's sanitizer allowlist would let tags such as `<a>` through.
- If `comment.deletable_by?(Current.user)` : `button_to "Delete", event_comment_path(comment.event_id, comment), method: :delete, class: "button button--ghost", form: { data: { turbo_confirm: "Delete this comment?" } }` .

Helper: `app/helpers/application_helper.rb`

Add `comment_time(comment)` , which returns `comment.created_at.strftime("%-d %B %Y . %H:%M")` to match `event_time` . If the team prefers relative times, use `time_ago_in_words` instead (see Outstanding questions).

Styles

Add `.comments` , `.comments__list` and `.comment` rules next to the existing `.attendees` and `.people` styles, reusing `muted` , `small` , `badge` and `button--ghost` .

Factory: `spec/factories/comments.rb` **(new)**

`factory :comment with association :event, association :user and body { "Is there parking nearby?" }` .

Infrastructure changes

There are no infrastructure changes. This feature adds:

- no queues or Active Job jobs
- no mailers (email notifications are out of scope)
- no external services
- no Action Cable or broadcasting

Feature flags: the repository has no feature flag system, and this plan doesn't add one. The feature goes live when the code is deployed. If a staged launch is needed, the team has to decide how (see Outstanding questions).

Cache store (only if rate limiting is used): `config.cache_store` is commented out in `config/environments/production.rb`. Rails' built-in `rate_limit` keeps its counters in `Rails.cache`. Without a shared cache store, limits apply per server, or not at all. In test, `:null_store` turns rate limiting off. If the team adopts `rate_limit`, a production cache store must be configured first. Solid Cache is one option, but it is not in the `Gemfile` today.

Rollout order:

1. Merge the migration, model and associations. This is safe to deploy alone, since nothing reads the table yet.
2. Merge the controller, routes and views. This makes the feature visible.
3. Steps 1 and 2 can ship in one deploy. Splitting them only lets the database step be reviewed on its own.

Work overview

Out of Scope

Replies and threads, editing a comment, email notifications, a moderation queue.

Work items

#	Title	Type	Kind	Estimate	Depends	Issue
T1	Migration: Create comments table	TASK	migration	2	-	#34
T2	Add Comment model and delete comments with their event or author	TASK	code	2	T1	#35
T3	Show comments on the event page	STORY	code	3	T2	#36
T4	Post a comment on an event	STORY	code	3	T3	#37
T5	Let authors delete their own comments	STORY	code	3	T4	#38
T6	Let organizers delete any comment on their event	STORY	code	2	T5	#39

Estimated total: 15 points

T1. Migration: Create comments table

Add one migration that creates an empty `comments` table. It stores one comment per row: which event it is on, who wrote it, and the text. The database enforces the rules on its own, before any application code reads or writes the table. No existing table changes. Old code never reads `comments`, so this migration can deploy before the application code or with it. Rolling back drops the table and every comment in it.

ACCEPTANCE CRITERIA

1. Migrating creates the comments table with NOT NULL columns `event_id`, `user_id`, `body`, `created_at` and `updated_at`, and `body` has no default
2. Inserting a comments row for an event or user that does not exist fails with `ActiveRecord::InvalidForeignKey`
3. Inserting a comments row with an empty or space-only body fails the check constraint `comments_body_length_check`
4. Inserting a comments row with a 1,001-character body fails the check constraint `comments_body_length_check`, and a 1,000-character body containing accented letters and emoji is accepted
5. The index `index_comments_on_event_id_and_created_at` is on `event_id`, `created_at`, the index `index_comments_on_user_id` is on `user_id`, and there is no index on `event_id` alone
6. Neither comments foreign key cascades on delete: deleting an event or user that has a comments row directly in the database fails with `ActiveRecord::InvalidForeignKey`
7. Rolling back the comments migration drops the comments table and leaves the events, users, rsvps and sessions tables unchanged

IMPLEMENTATION NOTES

Generate with `bin/rails g model Comment event:references user:references body:text`, then keep only the migration. Leave out the model, spec and fixture files, because T2 adds them.

Existing migrations use `ActiveRecord::Migration[8.1]`, not 8.0. Follow

`db/migrate/20260929214624_create_rsvps.rb`:

```
create_table :comments do |t|
  # The [event_id, created_at] index already leads with event_id.
  t.references :event, null: false, foreign_key: true, index: false
  t.references :user, null: false, foreign_key: true
  t.text :body, null: false
  t.timestamps
  t.index [ :event_id, :created_at ]
  t.check_constraint "length(trim(body)) > 0 AND length(body) <= 1000", name: "commer
end
```

The foreign keys have no `on_delete`, because cascades belong to the application (T2). Commit the updated `db/schema.rb`. Add `spec/db/comments_table_spec.rb` in the style of `spec/db/rsvps_table_spec.rb`, plus table helpers in `spec/support/` and `features/support/` like `rsvps_table_helpers.rb` and `rsvps_table.rb`. Write the scenarios in `features/pd-3-comments-on-events/t1.feature` with tag `@pd-3-t1`, and the steps in `features/step_definitions/comments_table_steps.rb`, following `features/pd-1-rsvps-with-a-waitlist/t1.feature`.

Touches: comments

T2. Add Comment model and delete comments with their event or author

Add the `Comment` model with its validations, ordering scope and factory. Connect it to `Event` and `User` so that deleting an event or a user also deletes the related comments. Without this, the new foreign keys would block deletes: `User` destroys its `organized_events`, and the foreign keys would stop the events or users from being removed. The feature isn't visible to users yet, because there are no routes or views.

ACCEPTANCE CRITERIA

1. A comment with an event, an author and a body is valid
2. A comment whose body is blank or only whitespace, including newlines and tabs, is invalid, because the body is stripped before validation
3. A comment of exactly 1,000 characters is valid, and one of 1,001 characters is invalid with the error 'Body is too long (maximum is 1000 characters)'
4. Comments on an event are listed oldest first, and two comments with the same `created_at` are listed in id order
5. Destroying an event deletes its comments
6. Destroying a user deletes the comments they wrote on other people's events
7. Destroying a user who organizes an event that has other people's comments succeeds without a foreign key error, and those comments are gone

IMPLEMENTATION NOTES

New file `app/models/comment.rb`:

```
class Comment < ApplicationRecord
  MAX_LENGTH = 1_000
  belongs_to :event
  belongs_to :user
  validates :body, presence: true, length: { maximum: MAX_LENGTH }
  normalizes :body, with: ->(body) { body.strip }
  scope :oldest_first, -> { order(:created_at, :id) }
end
```

`MAX_LENGTH` must be 1_000 so it matches `comments_body_length_check`. The plan's model snippet says 2_000, but its Database section overrides that. In `app/models/event.rb`, add `has_many :comments, dependent: :delete_all, inverse_of: :event` next to `has_many`

`:rsvps`. In `app/models/user.rb`, add `has_many :comments, dependent: :delete_all`. Comment has no callbacks, so `delete_all` is safe. Add `spec/factories/comments.rb` with `association :event`, `association :user` and `body { "Is there parking nearby?" }`. Put specs in `spec/models/comment_spec.rb`, `spec/models/event_spec.rb` and `spec/models/user_spec.rb`, and scenarios in `features/pd-3-comments-on-events/t2.feature`. `deletable_by?` is left out on purpose: T5 and T6 add it.

Touches: comments, events, users

T3. Show comments on the event page

I want to read the comments on an event page, with who wrote each one and when, so that I can find answers to questions other people have already asked

Show a Comments section on the event page, directly after the RSVP card. Anyone can see it, including signed-out visitors. The section has the heading 'Comments (N)' and the comments oldest first. Each comment shows the author's name, when it was posted and the text. If there are no comments, it shows 'No comments yet. Ask a question.' Comment text is always shown as plain, escaped text, with line breaks kept. For older users whose name is blank, the author shows as 'Someone'. The page must load comments and their authors in a fixed number of queries, however many comments there are. This ticket adds no posting form and no delete button: T4 adds the form, and T5 and T6 add deleting.

ACCEPTANCE CRITERIA

1. A signed-out visitor opening an event with comments sees the heading 'Comments (2)' and each comment's author name, posted time and body
2. Comments are listed oldest first
3. An event with no comments shows 'Comments (0)' and 'No comments yet. Ask a question.'
4. The posted time is shown as '30 September 2026 · 15:19' inside a time element whose `datetime` attribute is the ISO 8601 timestamp
5. A comment whose author has a blank name, set with `update_column`, shows the author as 'Someone'
6. A comment body containing `<script>alert(1)</script>` and `<a href="javascript:alert(1)">x</a>` is shown as literal escaped text, and no script or link element is rendered
7. A comment body with line breaks is shown as separate lines or paragraphs
8. The page makes 1 query against comments, and the number of queries against users is the same for 1 comment as for 10 comments by different authors

IMPLEMENTATION NOTES

In `app/controllers/events_controller.rb` `#show`, add `@comments = @event.comments.oldest_first.includes(:user)`. In `app/views/events/show.html.erb`, add `<%= render "events/comments" %>` right after `<%= render "events/rsvp_card" %>`, inside `aside#event-facts`. Placement is still Outstanding question 1: confirm it with Ana before merging, and move the render line if the answer differs. New partial `app/views/events/_comments.html.erb`: `<section class="section comments" id="comments">`, then `<h2>Comments (<%= @comments.size %>)</h2>`. Use `.size`, not `.count`,

so no extra query runs. Then `<ol class="comments__list" id="comments-list">` with `render partial: "comments/comment", collection: @comments`, or `<p class="muted small">`No comments yet. Ask a question.`</p>` when there are none. New partial `app/views/comments/_comment.html.erb`: `<li id="<%= dom_id(comment) %%" class="comment">` containing `comment.user.name.presence || "Someone"`, `<time datetime="<%= comment.created_at.iso8601 %%"><%= comment_time(comment) %"></time>` and `simple_format(h(comment.body))`. The explicit `h` is required, and you must never use `raw` or `html_safe` on the body. In `app/helpers/application_helper.rb`, add `comment_time(comment)` returning `comment.created_at.strftime("%-d %B %Y · %H:%M")`, to match `event_time`. In `app/assets/stylesheets/application.css`, add `.comments`, `.comments__list` and `.comment` next to `.attendees` and `.people`. Put request specs in `spec/requests/events_spec.rb`. Scenarios go in `features/pd-3-comments-on-events/t3.feature`. Reuse the query-counting steps from `features/pd-1-rsvps-with-a-waitlist/t5.feature` ('When I visit ... while counting queries', 'N query ran against `<table>`').

Touches: comments, users

T4. Post a comment on an event

I want to post a comment on an event page, so that I can ask the organizer and other attendees a question or share a link where the answer stays visible

Signed-in users see a comment form in the Comments section. Submitting it sends `POST /events/:event_id/comments`, which creates a comment written by the current user on that event. The request then redirects back to the event's Comments section with the notice 'Comment posted.' If validation fails, it redirects back with the error as an alert and creates nothing. Signed-out visitors see a 'Sign in to comment' link instead of the form. If they post anyway, they are redirected to sign-in. The author and the event always come from the session and the URL, never from form params.

ACCEPTANCE CRITERIA

1. A signed-in user who submits 'Is there parking nearby?' is taken back to the event's comments section and sees 'Comment posted.' and the comment with their name
2. A signed-in user sees the comment form, and the textarea is required and has maxlength 1000
3. A signed-out visitor sees a 'Sign in to comment' link and no comment form, and following the link opens the sign-in page
4. A signed-out POST to the event's comments is redirected to the sign-in page, and no comment is created
5. Posting a blank or whitespace-only body creates no comment and shows the alert "Body can't be blank"
6. Posting a 1,001-character body creates no comment and shows the alert 'Body is too long (maximum is 1000 characters)'
7. Posting with extra `user_id`, `event_id` and `created_at` params creates a comment owned by the signed-in user on the event from the URL, with its own timestamp
8. Posting to an event that does not exist returns 404

IMPLEMENTATION NOTES

In `config/routes.rb`, add `resources :comments, only: [:create]` inside `resources :events`. T5 adds `:destroy`. New file `app/controllers/comments_controller.rb`, following `RsvpsController`: it inherits `require_authentication` and has no `allow_unauthenticated_access`. Add `before_action :set_event` with `@event = Event.find(params[:event_id])`. `create` does `@comment = @event.comments.new(comment_params.merge(user: Current.user))`. On save, it runs `redirect_to event_path(@event, anchor: "comments")`, notice: "Comment posted.". Otherwise it runs the same redirect with `alert: @comment.errors.full_messages.to_sentence`. `comment_params` is `params.expect(comment: [:body])`. In `app/views/events/_comments.html.erb`, when `authenticated?`, add `form_with` model: `[@event, Comment.new]`, id: "new-comment" with `text_area :body`, `required: true`, `maxlength: Comment::MAX_LENGTH` and a submit button using `data: { turbo_submits_with: "Posting..." }`. Use `Comment.new`, not `@event.comments.build`, which would add an unsaved row to `@comments`. Otherwise, show `link_to "Sign in to comment"`, `new_session_path`, class: "button". Leave `rate_limit` out. Outstanding question 9 is unresolved, and production has no cache store. Put request specs in `spec/requests/comments_spec.rb`, mirroring the RSVP spec 'ignores user_id and status params'. Scenarios go in `features/pd-3-comments-on-events/t4.feature`.

Touches: comments

T5. Let authors delete their own comments

I want to delete a comment I posted, so that I can remove a question that was answered or a mistake I made

Add `Comment#deletable_by?(user)`, which is true only for the comment's author for now, and add `DELETE /events/:event_id/comments/:id`. The author sees a Delete button on their own comments, with a confirmation prompt. Confirming hard-deletes the comment and redirects back with 'Comment deleted.' Authorization runs in the controller, whether or not the button is shown. Anyone else who sends the request gets 'You can't delete this comment.' and the comment stays. The comment lookup is scoped to the event in the URL, so a comment id from another event returns 404. Each deletion is logged.

ACCEPTANCE CRITERIA

1. An author clicks Delete on their own comment, confirms 'Delete this comment?', sees 'Comment deleted.', and the comment is gone
2. A signed-in user sees no Delete button on comments written by other people
3. A signed-in user who is neither author nor organizer and sends DELETE for someone else's comment gets 'You can't delete this comment.', and the comment remains
4. A signed-out DELETE is redirected to the sign-in page, and the comment remains
5. Sending DELETE for a comment id through a different event's URL returns 404, and the comment remains
6. Deleting a comment writes the log line 'Comment deleted: comment_id=<id> event_id=<event_id> author_id=<author_id> deleted_by=<current user id>'

7. Showing Delete buttons adds no queries against events: the page still makes 1 query against comments, and the number of queries against events is the same for 1 comment as for 10

IMPLEMENTATION NOTES

In `app/models/comment.rb`, add `def deletable_by?(user) = user.present? && user_id == user.id`. T6 extends it to organizers. In `config/routes.rb`, change comments to `only: %i[create destroy]`. `CommentsController#destroy`: `comment = @event.comments.find(params[:id])`. If `comment.deletable_by?(Current.user)` is false, return `redirect_to event_path(@event), alert: "You can't delete this comment."`, which mirrors `EventsController#require_organizer`. Otherwise run `comment.destroy!`, then `Rails.logger.info("Comment deleted: comment_id=#{comment.id} event_id=#{comment.event_id} author_id=#{comment.user_id} deleted_by=#{Current.user.id}")`, then `redirect_to event_path(@event, anchor: "comments"), notice: "Comment deleted."`. In `app/views/comments/_comment.html.erb`, add button to "Delete", `event_comment_path(comment.event_id, comment)`, `method: :delete`, `class: "button button-ghost"`, `form: { data: { turbo_confirm: "Delete this comment?" } }` when `comment.deletable_by?(Current.user)`. Pass `comment.event_id`, not `comment.event`, to avoid loading the event per comment. Put specs in `spec/models/comment_spec.rb` (`author: true`; other `user: false`; `nil: false`) and `spec/requests/comments_spec.rb`. Scenarios go in `features/pd-3-comments-on-events/t5.feature`.

Touches: comments

T6. Let organizers delete any comment on their event

I want to delete any comment on an event I organize, so that I can remove spam or off-topic posts from my event page

Extend `Comment#deletable_by?` so the event's organizer can also delete any comment on that event, using the existing `Event#organized_by?`. Organizers see a Delete button on every comment on their own events. Organizers of other events get no extra rights. The organizer check must use the event that is already loaded on the page, so rendering the buttons runs no query per comment.

ACCEPTANCE CRITERIA

1. An organizer clicks Delete on another person's comment on their event, confirms, sees 'Comment deleted.', and the comment is gone
2. An organizer sees a Delete button on every comment on their event
3. The organizer of a different event sees no Delete button on comments by others, and sending DELETE gets 'You can't delete this comment.' while the comment remains
4. An organizer deleting someone else's comment writes a log line where `deleted_by` is the organizer's id and `author_id` is the comment author's id
5. Viewed by the organizer, an event with 10 comments makes 1 query against comments, and the number of queries against events and users is the same as with 1 comment

IMPLEMENTATION NOTES

In `app/models/comment.rb`: `def deletable_by?(user) = user.present? && (user_id == user.id || event.organized_by?(user))`. `@event.comments.oldest_first` is an association

relation, and with `inverse_of: :event` (added in T2) each comment's `event` points to the already-loaded `@event`. If query counting shows a query per comment, change `EventsController#show` to `includes(:user, :event)`. Model spec cases: the event's organizer returns true, and the organizer of a different event returns false. Put request specs in `spec/requests/comments_spec.rb` and `spec/requests/events_spec.rb`. Scenarios go in `features/pd-3-comments-on-events/t6.feature`. Optional: an 'Organizer' badge on the organizer's own comments is Outstanding question 11 and not part of this ticket.

Touches: comments, events

Risks

- **Stored XSS through comment bodies.** Comments are user input shown to every visitor, including guests.
 - *Mitigation:* render with `simple_format(h(comment.body))`, and never use `raw` or `html_safe` on the body.
 - *Mitigation:* a request spec posts `<script>` and `<a href="javascript:...">` and asserts the response shows them escaped.
 - *Mitigation:* Brakeman already runs (`gem "brakeman"`).
- **Deleting someone else's comment, or a comment on another event.**
 - *Mitigation:* `@event.comments.find` limits the lookup to the event in the URL.
 - *Mitigation:* `Comment#deletable_by?` allows only the author or `event.organized_by?`.
 - *Mitigation:* request specs cover a user who is neither author nor organizer, and a comment id from another event.
- **Foreign key failures when a user or event is deleted.** User destroys `organized_events`, and the new `comments.user_id` and `comments.event_id` foreign keys would block that.
 - *Mitigation:* `dependent: :delete_all` on both `Event#comments` and `User#comments`.
 - *Mitigation:* model specs cover destroying a user who wrote comments and organizes events that have comments.
- **Spam or flooding.** Anyone who can sign up can post, and there is no moderation queue (out of scope).
 - *Mitigation:* organizers can delete any comment on their event.
 - *Mitigation:* an optional `rate_limit` on `create`, once a production cache store exists.
 - *Mitigation:* a body length cap.
- **Blank author names on legacy users.** The `users.name` column defaults to `""`.
 - *Mitigation:* the partial falls back to `Someone`, and a spec covers it.
- **Lost input when validation fails.** The controller redirects instead of re-rendering the page.
 - *Mitigation:* `required` and `maxlength` on the textarea stop the two failure cases (blank and too long) in the browser.
 - *Accepted:* in the rare case of a server-side failure, the typed text is lost. This is a deliberate simplification that avoids duplicating the `EventsController#show` setup.
- **Public exposure of names.** Today, attendee names are shown only to the organizer (`@rsvps` is organizer-only). Comment author names would be visible to anonymous visitors.

- *Mitigation*: needs a product decision before release (see Outstanding questions).
- **Unbounded list on popular events.**
- *Mitigation*: acceptable at current scale. Revisit with pagination if an event goes beyond a few hundred comments (see Performance).

Performance

- **Event page query count:** `events#show` runs two new queries, however many comments there are:
 - `SELECT comments.* FROM comments WHERE event_id = ? ORDER BY created_at, id`
 - one `SELECT users.* WHERE id IN (...) from includes(:user)`
- **N+1 risks, and how the plan avoids them:**
 - Author names: `includes(:user)` preloads `comment.user.name`.
 - The delete button check: `comment.deletable_by?` calls `comment.event.organized_by?`. Done carelessly, this loads the event once per comment. The partial passes `comment.event_id` to the route helper, and `deletable_by?` should use the event that is already loaded. Get this by adding `inverse_of: :event` on `Event` has_many `:comments` (Rails infers it here), or by preloading with `includes(:user, :event)`. A request spec or Cucumber step should assert the number of queries against `comments` and `users`, in the style of the existing step `1 query ran against rsvps` in `features/pd-1-rsvps-with-a-waitlist/t5.feature`.
 - The heading count uses `@comments.size` on the loaded relation, not `.count`.
- **Indexes:**
 - `[event_id, created_at]` covers both the filter and the sort in the page query.
 - `user_id` covers `user.comments.delete_all`.
 - `destroy` looks up `comments.id`, the primary key.
- **Large tables:** none are affected. `comments` starts empty, and `events` and `users` are not altered.
- **Backfill:** none, so no batch size applies.
- **Unbounded rendering:** v1 renders every comment on an event. At the expected volume (tens per event) this is fine. If an event reaches roughly 200 or more comments, add pagination or a `limit` with a `Show older` link. That decision is left for later.
- **Deletes:** dependent: `:delete_all` issues one `DELETE` per association instead of loading the records.

Security

- **Authentication:**
 - `CommentsController` keeps `require_authentication` from `ApplicationController`. Guests who send a `POST` or `DELETE` are redirected to `new_session_path`, and nothing changes.
 - Reading comments stays public through the existing `allow_unauthenticated_access` only: `%i[ index show ]` in `EventsController`.
- **Authorization:**
 - Only the comment's author (`comment.user_id == Current.user.id`) or the event's organizer (`@event.organized_by?(Current.user)`) can delete a comment. `Comment#deletable_by?`

checks this.

- The check runs in the controller. Hiding the button in the view is only a convenience.
- The lookup is `@event.comments.find(params[:id])`, so a comment cannot be deleted through another event's URL.
- Editing is out of scope, so there is no update path to secure.
- **Mass assignment:**
 - `params.expect(comment: [ :body ])` permits only `body`.
 - `user` comes from `Current.user`, and `event` comes from the URL.
 - A spec posts `user_id`, `event_id` and `created_at` and asserts they are ignored, mirroring the existing RSVP spec ignores `user_id` and `status` params.
- **Input validation:**
 - `body` must be present after `strip` and at most `Comment::MAX_LENGTH` characters long.
 - The database enforces `NOT NULL`, and optionally a non-blank check constraint.
- **Output encoding and XSS:** the body is rendered with `simple_format(h(comment.body))`. Links show as plain text in v1. Whether they should be clickable is an Outstanding question. Any auto-linking must allow only `http` and `https` and add `rel="nofollow noopener ugc"`.
- **CSRF:** Rails' default forgery protection covers `form_with` and `button_to`. No API endpoint is added.
- **Data exposure:**
 - The comment author's `name` and the posting time become visible to anyone, including guests.
 - `email_address` is never rendered.
 - This is a change from today, where attendee names are organizer-only. It needs sign-off (see Outstanding questions).
- **Abuse:** there is no moderation queue, by design. Organizer deletion and an optional rate limit are the controls.

Risk Level: MEDIUM. The feature adds public, user-written content that anonymous visitors can see, which carries XSS, spam and name-exposure risk, although the authorization rule itself (author or organizer) is simple.

Monitoring

The repository has no APM or error tracker configured, so these checks rely on Rails logs and whatever log aggregation production uses. Confirm the tool with DevOps.

- **Errors:**
 - Any 5xx from `CommentsController#create` or `#destroy`, or from `EventsController#show` after release. A regression in `show` would break every event page.
 - `ActiveRecord::InvalidForeignKey` when deleting users or events. This would mean a `dependent:` option is missing.
- **Request outcomes:**
 - How often `POST /events/:event_id/comments` ends in a redirect with an alert (validation failures).

- `DELETE` requests that end in `You can't delete this comment.` A spike suggests someone is probing.
- 429 responses, if `rate_limit` is enabled.
- **Logs:** the `Comment deleted: comment_id=... event_id=... author_id=... deleted_by=...` lines. Watch for organizers deleting many comments at once, which points to spam.
- **Metrics** (ad hoc SQL or console queries for the first few weeks):
 - comments per day
 - comments per event
 - the number of distinct commenters
 - the share of comments deleted by organizers rather than by their authors
- **Performance:** `events#show` response time before and after release. Expect little change, since only two indexed queries are added.
- **Jobs:** none. This feature adds no background work.

Outstanding questions

1. **Placement:** "Below the RSVP card" puts comments inside the narrow sidebar `aside.event__facts`, above the organizer's **Edit event** link and the attendee lists. Should comments go there, in the main column below **About this event**, or full-width under `.event__layout`? This must be answered before the UI ticket.
2. **Links:** the brief says people share links. Should URLs be clickable, or is plain text acceptable for v1? Clickable links need `rails_autolink` or a small helper, with `rel="nofollow noopener ugc"`.
3. **Who may comment:** any signed-in user, or only people with an RSVP (going or waitlisted) plus the organizer?
4. **Past events:** can people comment after `starts_at`, when RSVPs are no longer possible? Should comments stay visible on past events?
5. **Maximum length:** is 2,000 characters acceptable?
6. **Public names:** is it acceptable to show commenter names to signed-out visitors? Today attendee names are shown only to the organizer.
7. **Timestamp format:** absolute (`30 September 2026 · 15:19`, matching `event_time`) or relative (`5 minutes ago`)? And in which time zone? None of the files reviewed sets one explicitly.
8. **Hard delete:** is permanent deletion fine, or do organizers or the team need a record of deleted comments (soft delete with `deleted_at`)? A moderation queue is out of scope, but an audit trail may still be wanted.
9. **Rate limiting:** should `create` be rate limited in v1? If so, which production cache store should be used? `config.cache_store` is currently unset in `config/environments/production.rb`.
10. **Staged launch:** is it acceptable for the feature to go live on deploy, given there is no feature flag system?
11. **Organizer as author:** should the organizer's comments be marked, for example with an `Organizer` badge, so their answers stand out?

Testing

Model specs use RSpec (`spec/models`) and request specs live in `spec/requests` . End-to-end scenarios are Cucumber features in `features/pd-3-comments-on-events/` , with steps in `features/step_definitions/` , following the `pd-1` layout.

Model: `spec/models/comment_spec.rb`

- Valid with an event, a user and a body.
- Invalid with a blank or whitespace-only body. `normalizes` strips the body first.
- Invalid when the body is longer than `Comment::MAX_LENGTH` . Valid at exactly the limit.
- `oldest_first` orders by `created_at` , then by `id` when timestamps are equal.
- `deletable_by?` :
 - true for the author
 - true for the event's organizer
 - false for another signed-in user
 - false for `nil`
 - false for the organizer of a different event

Model: `spec/models/event_spec.rb` **and** `spec/models/user_spec.rb`

- Destroying an event deletes its comments.
- Destroying a user deletes the comments they wrote on other people's events.
- Destroying a user who organizes an event with other people's comments succeeds without a foreign key error.

Database: `spec/db/comments_table_spec.rb` (**following** `spec/db/rsvps_table_spec.rb`)

- `event_id` , `user_id` and `body` reject `NULL` .
- The foreign keys reject unknown events and users.
- The `[event_id, created_at]` index exists.
- If adopted, the check constraint rejects a blank body.

Request: `spec/requests/comments_spec.rb`

- **Guest:**
 - `POST` redirects to `new_session_path` and creates nothing.
 - `DELETE` redirects to sign-in and deletes nothing.
- **Signed in:**
 - `POST` with a body creates a comment owned by `Current.user` and redirects to `event_path(event, anchor: "comments")` with `Comment` posted.
 - `POST` with a blank body creates nothing and sets an alert.
 - `POST` ignores the `user_id` , `event_id` and `created_at` params.
 - `POST` to an unknown event returns 404.
- **Delete permissions:**

- The author can delete their own comment.
- The organizer can delete anyone's comment on their event.
- Another user gets `You can't delete this comment.` and the comment remains.
- The organizer of a different event cannot delete it.
- A comment id that belongs to another event returns 404.
- **XSS:** posting `<script>alert(1)</script>` shows escaped text on `GET event_path`.

Request: `spec/requests/events_spec.rb`

- A guest sees the comments and a `Sign in to comment` link, and no form.
- A signed-in user sees the form.
- The delete button appears only for the author and the organizer.
- **Legacy data:** a comment by a user whose `name` is `""` (set with `update_column`) shows the author as `Someone`.

Cucumber: `features/pd-3-comments-on-events/*.feature`

- A signed-in person posts a question and sees it with their name and the time.
- Comments are listed oldest first.
- An author deletes their own comment after confirming.
- An organizer deletes someone else's comment.
- A user who is not the organizer sees no delete button on other people's comments.
- A guest can read comments but is sent to sign-in to comment.
- The event page loads comments and authors in a fixed number of queries, whatever the comment count: one against `comments` and one against `users`.

Static checks

`bin/brakeman` and `bin/rubocop` pass.

Sign-off

Role	Decision	By	When
Review	approved	Ivan Blažević <ivan.blazevic@rubycode.co>	30 Sep 2026 13:25